

Assignment II - Valgrind & Pytorch profiler

111550129, 林彦亨

1. Memcheck

Error 1 Invalid write — Out-of-bounds Write

Cause : `malloc(40)` in line 46 allocates space for 10 integers (40 bytes). However, `memleak` writes to the 11th element (index 10), which is 0 bytes past the end of the allocated block. This is an out-of-bounds write caused by an off-by-one error.

Error 2 Invalid read — Out-of-bounds Read

Cause : The program reads from the same out-of-bounds address as Error 1. This is an out-of-bounds read, typically caused by the same off-by-one logic error that led to Error 1.

Error 3 Use of Uninitialised Value — Uninitialized variable passed to `printf`

Cause : `printf()` is called with a variable that was declared but never assigned a value. Since `printf` internally accesses this uninitialized value in multiple places during formatting.

Error 4 Fishy malloc Argument — Negative size passed to `malloc`

Cause : `malloc(-40)` is called with a negative size argument. Since `malloc` expects an unsigned `size_t` value, passing a negative integer causes an integer overflow, resulting in an extremely large allocation size. This indicates a logical error in computing the allocation size.

Error 5 Invalid free() — Double Free

Cause : The block allocated at line 63 was already freed at line 64. At line 65, `free()` is called again on the same pointer, resulting in a double free. This corrupts the heap allocator's internal state and can lead to crashes or security vulnerabilities.

Error 6 Memory Leak — Definitely Lost

Cause : The 40-byte block (10 integers) allocated by `malloc()` at line 46 is never freed throughout the program's execution. When the program exits, the memory is still in use and no pointer to it exists, so it can never be freed.

2. Cachegrind

111550129_good_log

```
1  ==56647== Cachegrind, a cache and branch-prediction profiler
2  ==56647== Copyright (C) 2002-2017, and GNU GPL'd, by Nicholas Nethercote et al.
3  ==56647== Using Valgrind-3.20.0 and LibVEX; rerun with -h for copyright info
4  ==56647== Command: ./good
5  ==56647== Parent PID: 18369
6  ==56647==
7  --56647-- warning: L3 cache found, using its data for the LL simulation.
8  --56647-- warning: specified LL cache: line_size 64  assoc 12  total_size 37,748,736
9  --56647-- warning: simulated LL cache: line_size 64  assoc 18  total_size 37,748,736
10 ==56647==
11 ==56647== I   refs:      30,193,736
12 ==56647== I1  misses:      1,089
13 ==56647== L1i misses:      1,073
14 ==56647== I1  miss rate:      0.00%
15 ==56647== L1i miss rate:      0.00%
16 ==56647==
17 ==56647== D   refs:      14,058,814 (12,045,511 rd  + 2,013,303 wr)
18 ==56647== D1  misses:      126,552 (    63,725 rd  +    62,827 wr)
19 ==56647== L1d misses:      63,851 (    1,042 rd  +    62,809 wr)
20 ==56647== D1  miss rate:      0.9% (    0.5%  +    3.1% )
21 ==56647== L1d miss rate:      0.5% (    0.0%  +    3.1% )
22 ==56647==
23 ==56647== LL refs:      127,641 (    64,814 rd  +    62,827 wr)
24 ==56647== LL misses:      64,924 (    2,115 rd  +    62,809 wr)
25 ==56647== LL miss rate:      0.1% (    0.0%  +    3.1% )
```

111550129_bad_log

```
1  ==56849== Cachegrind, a cache and branch-prediction profiler
2  ==56849== Copyright (C) 2002-2017, and GNU GPL'd, by Nicholas Nethercote et al.
3  ==56849== Using Valgrind-3.20.0 and LibVEX; rerun with -h for copyright info
4  ==56849== Command: ./bad
5  ==56849== Parent PID: 18369
6  ==56849==
7  --56849-- warning: L3 cache found, using its data for the LL simulation.
8  --56849-- warning: specified LL cache: line_size 64  assoc 12  total_size 37,748,736
9  --56849-- warning: simulated LL cache: line_size 64  assoc 18  total_size 37,748,736
10 ==56849==
11 ==56849== I   refs:      30,193,722
12 ==56849== I1  misses:      1,075
13 ==56849== L1i misses:      1,073
14 ==56849== I1  miss rate:      0.00%
15 ==56849== L1i miss rate:      0.00%
16 ==56849==
17 ==56849== D   refs:      14,058,806 (12,045,503 rd  + 2,013,303 wr)
18 ==56849== D1  misses:      2,001,616 ( 1,001,285 rd  + 1,000,331 wr)
19 ==56849== L1d misses:      63,851 (    1,042 rd  +    62,809 wr)
20 ==56849== D1  miss rate:      14.2% (    8.3%  +   49.7% )
21 ==56849== L1d miss rate:      0.5% (    0.0%  +    3.1% )
22 ==56849==
23 ==56849== LL refs:      2,002,691 ( 1,002,360 rd  + 1,000,331 wr)
24 ==56849== LL misses:      64,924 (    2,115 rd  +    62,809 wr)
25 ==56849== LL miss rate:      0.1% (    0.0%  +    3.1% )
```

The key difference between good and bad

During the ascending phase (0 → 65), the program continuously allocates new heap memory without releasing it until it returns, causing the heap usage to accumulate steadily; during the descending phase (66 → 75), the functions begin to return and release their allocated memory. The heap usage gradually decreases as each layer of allocations is freed.

Allocated and used bytes at peak (snapshot 65)

n	time(B)	total(B)	useful-heap(B)	extra-heap(B)	stacks(B)
60	225,568	225,568	225,000	568	0
61	229,576	229,576	229,000	576	0
62	231,584	231,584	231,000	584	0
63	235,592	235,592	235,000	592	0
64	239,600	239,600	239,000	600	0
65	239,600	239,600	239,000	600	0

At the peak (snapshot 65), the program has allocated a total of 239,600 bytes, of which 239,000 bytes are actually used by the program, and the remaining 600 bytes are internal overhead from the heap allocator.

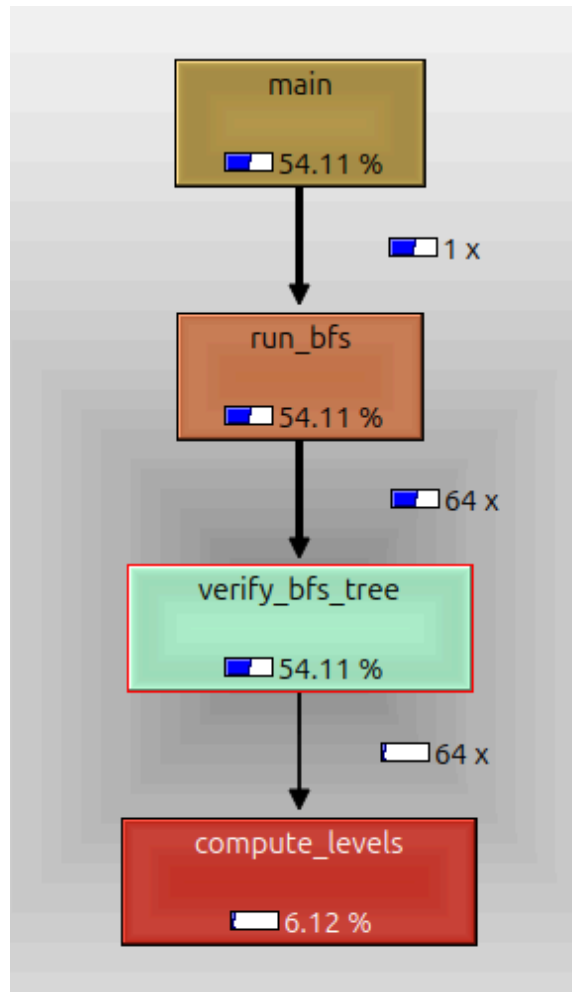
4. Callgrind

Most expensive function : `verify_bfs_tree`

Based on the Self cost column in `kcachegrind` GUI, the most expensive function excluding callee time is `verify_bfs_tree` (located in `seq-csr: verify.c`), with a self cost of 47.98%. It is called 64 times by `run_bfs`.

Incl.	Self	Called	Function	Location
54.11	47.98	64	<code>verify_bfs_tree</code>	<code>seq-csr: verify.c</code>
11.26	11.25	64	<code>make_bfs_tree</code>	<code>seq-csr: seq-csr.c</code>
6.12	6.12	64	<code>compute_levels</code>	<code>seq-csr: verify.c</code>
3.98	3.98	222 002	<code>mod_mac</code>	<code>seq-csr: mod_arith_64bit.h</code>
16.64	3.17	12 288	<code>make_one_edge</code>	<code>seq-csr: graph_generator.c</code>
11.04	2.78	24 274	<code>mrg_apply_transition</code>	<code>seq-csr: splittable_mrg.c</code>
2.65	2.65	173 454	<code>mod_mul</code>	<code>seq-csr: mod_arith_64bit.h</code>
11.49	2.62	122 880	<code>generate_4way_bernoulli</code>	<code>seq-csr: graph_generator.c</code>
8.23	2.57	124 906	<code>mrg_orig_step</code>	<code>seq-csr: splittable_mrg.c</code>
1.96	1.96	72 822	<code>mod_mac3</code>	<code>seq-csr: mod_arith_64bit.h</code>
1.65	1.65	72 822	<code>mod_mac2</code>	<code>seq-csr: mod_arith_64bit.h</code>
2.95	1.53	22 948	<code>msort_with_tmp.part.0'2</code>	<code>libc.so.6: qsort.c, string_fortified.h</code>
5.50	1.52	222 002	<code>mod_mac_y</code>	<code>seq-csr: mod_arith_64bit.h</code>
1.51	1.51	48 548	<code>mod_mac4</code>	<code>seq-csr: mod_arith_64bit.h</code>
1.37	1.37	132 383	<code>i64cmp</code>	<code>seq-csr: seq-csr.c</code>
1.11	1.11	49 152	<code>bitreverse</code>	<code>seq-csr: graph_generator.c</code>
9.02	0.79	124 906	<code>mrg_get_uint_orig</code>	<code>seq-csr: splittable_mrg.c</code>
1.89	0.78	24 576	<code>scramble</code>	<code>seq-csr: graph_generator.c</code>
2.57	0.66	124 906	<code>mod_mul_x</code>	<code>seq-csr: mod_arith_64bit.h</code>
11.74	0.51	12 289	<code>mrg_skip</code>	<code>seq-csr: splittable_mrg.c</code>
3.88	0.46	1 024	<code>pack_vtx_edges</code>	<code>seq-csr: seq-csr.c</code>
0.52	0.43	1	<code>setup_deg_off</code>	<code>seq-csr: seq-csr.c</code>
0.31	0.31	24 356	<code>scatter_edge</code>	<code>seq-csr: seq-csr.c</code>
0.27	0.27	34 579	<code>__memcpy_avx_unalign...</code>	<code>libc.so.6: memmove-vec-unaligned-erms.S</code>
28.63	0.25	1	<code>generate_kronecker_ra...</code>	<code>seq-csr: graph_generator.c</code>

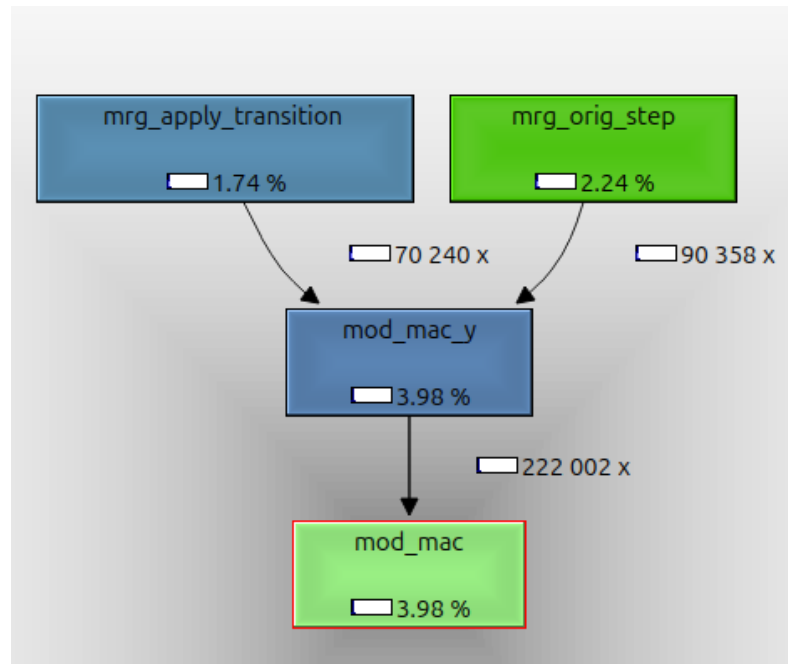
The call graph shows the execution path as `main` → `run_bfs` → `verify_bfs_tree`, where `verify_bfs_tree` dominates the overall execution time even when its callee cost is excluded.



Most frequently called function and its caller : `mod_mac` and `mod_mac_y`

Incl.	Self	Called	Function	Location
3.98	3.98	222 002	<code>mod_mac</code>	seq-csr: mod_arith_64bit.h
5.50	1.52	222 002	<code>mod_mac_y</code>	seq-csr: mod_arith_64bit.h
2.65	2.65	173 454	<code>mod_mul</code>	seq-csr: mod_arith_64bit.h
1.37	1.37	132 383	<code>i64cmp</code>	seq-csr: seq-csr.c
8.23	2.57	124 906	<code>mrg_orig_step</code>	seq-csr: splittable_mrg.c
9.02	0.79	124 906	<code>mrg_get_uint_orig</code>	seq-csr: splittable_mrg.c
2.57	0.66	124 906	<code>mod_mul_x</code>	seq-csr: mod_arith_64bit.h
11.49	2.62	122 880	<code>generate_4way_bernoulli</code>	seq-csr: graph_generator.c
1.96	1.96	72 822	<code>mod_mac3</code>	seq-csr: mod_arith_64bit.h
1.65	1.65	72 822	<code>mod_mac2</code>	seq-csr: mod_arith_64bit.h
0.18	0.18	49 155	<code>get_v0_from_edge</code>	seq-csr: graph_generator.h, graph_generator.h
0.18	0.18	49 155	<code>get_v1_from_edge</code>	seq-csr: graph_generator.h, graph_generator.h
1.11	1.11	49 152	<code>bitreverse</code>	seq-csr: graph_generator.c
1.51	1.51	48 548	<code>mod_mac4</code>	seq-csr: mod_arith_64bit.h
0.27	0.27	34 579	<code>__memcpy_avx_unalign...</code>	libc.so.6: memmove-vec-unaligned-erms.S
0.31	0.04	34 578	<code>0x000000000496a6b0</code>	(unknown)
1.89	0.78	24 576	<code>scramble</code>	seq-csr: graph_generator.c
0.31	0.31	24 356	<code>scatter_edge</code>	seq-csr: seq-csr.c
11.04	2.78	24 274	<code>mrg_apply_transition</code>	seq-csr: splittable_mrg.c
11.23	0.19	24 274	<code>mrg_step</code>	seq-csr: splittable_mrg.c
2.95	1.53	22 948	<code>msort_with_tmp.part.0'2</code>	libc.so.6: qsort.c, string_fortified.h

The most frequently called function is `mod_mac`, which is called 222,002 times. According to the call graph in `kcachegrind`, its caller is `mod_mac_y`, which invokes `mod_mac` all 222,002 times. `mod_mac_y` itself is called by two callers: `mrg_apply_transition` (70,240 times) and `mrg_orig_step` (90,358 times).



5. Pytorch profiler

The top three functions in terms of Self CPU time are:

```

• (base) heng@heng:~/Desktop/MSS_hw2$ python3 profiler.py
/home/heng/anaconda3/lib/python3.12/site-packages/torch/profiler/profiler.py:224: UserWarning: Warning: Profiler clears events at the end of each
cycle. Only events from the current cycle will be reported. To keep events across cycles, set acc_events=True.
  warn_once()

```

Name	Self CPU %	Self CPU	CPU total %	CPU total	CPU time avg	CPU Mem	Self CPU Mem	# of Calls
aten::addmm	29.90%	11.009ms	34.89%	12.846ms	132.430us	4.13 MB	4.13 MB	97
model_inference	23.83%	8.774ms	100.00%	36.821ms	36.821ms	0 B	-10.72 MB	1
aten::copy_	7.59%	2.795ms	7.59%	2.795ms	12.366us	0 B	0 B	226
aten::native_layer_norm	5.09%	1.874ms	5.79%	2.132ms	71.052us	939.66 KB	0 B	30
aten::view	3.12%	1.148ms	3.12%	1.148ms	2.849us	0 B	0 B	403
aten::bmm	2.88%	1.059ms	2.92%	1.075ms	29.858us	624.38 KB	624.38 KB	36
aten::einsum	2.63%	967.943us	14.94%	5.500ms	152.770us	2.32 MB	0 B	36
aten::softmax	2.44%	896.709us	2.44%	896.709us	49.817us	72.38 KB	72.38 KB	18
aten::permute	1.99%	732.543us	2.41%	888.948us	4.939us	0 B	0 B	180
aten::linear	1.93%	710.541us	40.44%	14.889ms	153.495us	4.13 MB	0 B	97
aten::empty	1.78%	653.999us	1.78%	653.999us	3.442us	3.24 MB	3.24 MB	190
aten::clamp_min	1.63%	600.568us	1.63%	600.568us	50.047us	1.50 MB	1.50 MB	12
aten::tril	1.56%	576.142us	1.56%	576.142us	576.142us	196 B	196 B	1
aten::reshape	1.32%	487.839us	8.04%	2.961ms	20.007us	2.25 MB	0 B	148
aten::add	1.31%	483.288us	1.31%	483.288us	15.103us	1000.00 KB	1000.00 KB	32
aten::as_strided	1.20%	442.556us	1.20%	442.556us	0.892us	0 B	0 B	496
aten::clone	1.11%	409.784us	6.00%	2.211ms	23.772us	2.32 MB	0 B	93
aten::unsqueeze	0.93%	343.038us	1.10%	404.776us	5.470us	0 B	0 B	74
aten::t	0.93%	340.639us	1.82%	669.207us	6.899us	0 B	0 B	97
aten::expand	0.87%	318.867us	1.19%	438.276us	3.223us	0 B	0 B	136

Self CPU time total: 36.821ms

Chrome trace saved to output.json

1. `aten::addmm` : Self CPU = 29.90% (11.009ms), called 97 times.
2. `aten::copy_` : Self CPU = 7.59% (2.795ms), called 226 times.
3. `aten::native_layer_norm` : Self CPU = 5.09% (1.874ms), called 30 times.

Two functions (colors) appear the most (in terms of time)

1. `aten::linear` (14.889ms) -> green
2. `aten::addmm` (12.846ms) -> pink

